Cletus — Section 1, Day 2

- **Date:** [YYYY-MM-DD]
- **Course:** Week 1, Day 2 — LLM building blocks, frontier vs open-source models, Chat Completions API
- **Lab notebook:** week1/day2.ipynb (solution: week1/solution.py)
- **Time spent:** [e.g. 2h]

Key concepts

- **Building blocks of LLM engineering** — *models* (closed frontier vs open-source), *tools* (Ollama, Hugging Face, Gradio, ...) and *techniques* (APIs, prompting, RAG, fine-tuning, agents). The 8 weeks cover them step by step.
- **Frontier (closed) models** — OpenAI GPT, Anthropic Claude, Google Gemini, xAI Grok. Strongest, and you pay per token.
- **Open-source (open-weight) models** — Meta Llama, Mistral, DeepSeek, Qwen, Gemma. Free to download; small ones run locally with Ollama.
- **Distillation** — a big model's knowledge squeezed into a small one. For example, `deepseek-r1:1.5b` is DeepSeek R1 distilled into a small Qwen model.
- **Chat Completions API** — "here is a conversation, predict what comes next". OpenAI invented it, and now nearly everyone uses it.
- **Endpoint** — a URL you send an HTTP request to. The `openai` Python package is just a thin client library around that HTTP call; it contains no model.
- **OpenAI-compatible endpoints** — Gemini, Ollama and others accept the same request format, so the same `OpenAI` client works. You only change `base_url` and `api_key`.

Code / examples

The same call two ways:

```
import os, requests

# 1) Raw HTTP request to the endpoint
headers = {"Authorization": f"Bearer {os.getenv('OPENAI_API_KEY')}", "Content-Type":
"application/json"}
payload = {"model": "gpt-5-nano", "messages": [{"role": "user", "content": "Tell me a fun fact"}]}
r = requests.post("https://api.openai.com/v1/chat/completions", headers=headers, json=payload)
print(r.json()["choices"][0]["message"]["content"])

# 2) Same thing with the client library
from openai import OpenAI
openai = OpenAI()
response = openai.chat.completions.create(model="gpt-5-nano", messages=payload["messages"])
print(response.choices[0].message.content)
```

Switching provider = new `base_url` + key, and the rest stays the same:

```
gemini = OpenAI(base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
                api_key=os.getenv("GOOGLE_API_KEY"))
ollama = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")  # any text works locally

ollama.chat.completions.create(model="llama3.2", messages=[{"role": "user", "content": "Tell me a
fun fact"}])
```

Ollama check: open http://localhost:11434 → "Ollama is running" (if not, run `ollama serve`). Download a model with `ollama pull llama3.2`.

Exercise / challenge

Homework: upgrade the Day 1 summarizer to use Llama 3.2 through Ollama instead of OpenAI. Only two lines change: the client (`base_url`) and the model name. - Compare the Llama summary with the GPT summary for the same site. - Local pros: no API cost, and data never leaves your machine. Con: much weaker than frontier models.

Questions & blockers

Watch out for: - `llama3.3` and `llama4` are too big for a laptop. Use `llama3.2`, or `llama3.2:1b` if it's slow. - Save `.env` after editing, then reload it with `load_dotenv(override=True)`. - Gemini keys start with `AIz` (or `AQ.`); get one free at https://aistudio.google.com/api-keys.

To discuss with my partner

- When would we pick a local open-source model over a frontier API (cost, privacy, quality)?
- How different were the Llama and GPT summaries?

Resources

- `guides/04_technical_foundations.ipynb` (endpoints, HTTP)
- `guides/09_ai_apis_and_ollama.ipynb` (Ollama, Gemini, OpenRouter code)
- Ollama: https://ollama.com